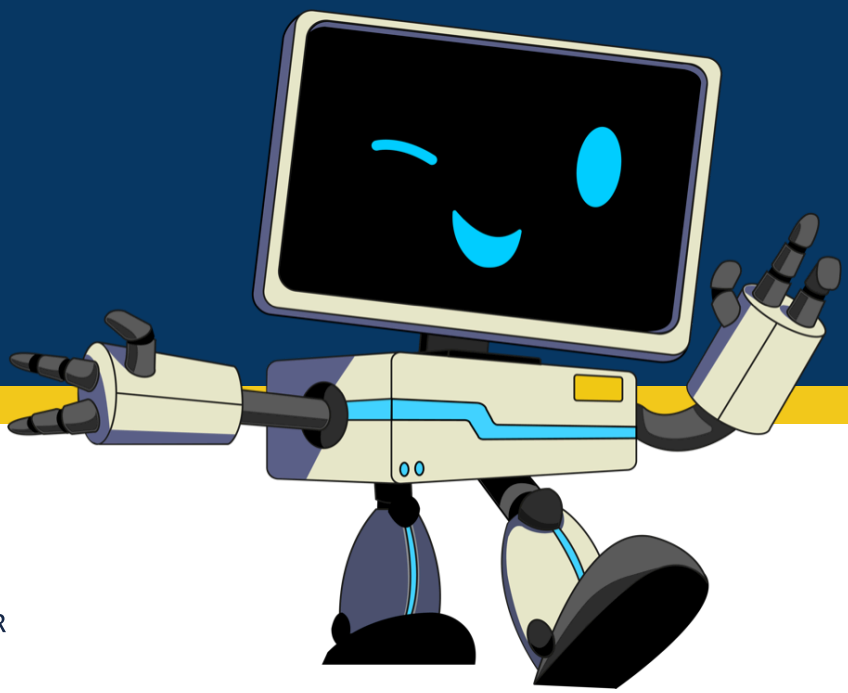
GUÍA DE REPASO · SESIÓN 3

Repetir sin repetirse: listas, bucles y funciones

Definiciones, ejemplos y ejercicios para repasar lo que vimos en clase.



CURSO LIBRE

Introducción a la Programación en Python y R

SEMESTRE

2026-2

ESTUDIANTE AUXILIAR

Yony Sebastián Chaparro Mesa

MATERIAL DEL CURSO

Moodle FCE

UIFCE
UNIDAD DE INFORMÁTICA

Unidad de Informática · Facultad de Ciencias Económicas
Universidad Nacional de Colombia, Sede Bogotá
uniic_bog@unal.edu.co

Cómo usar esta guía

Esta guía resume la tercera sesión del curso. Cada tema tiene **definiciones** (recuadros azules), **ejemplos de código** con su salida y **advertencias** sobre los errores más comunes (recuadros amarillos).

La mejor forma de repasar es con Google Colab abierto: **escribe tú mismo cada ejemplo**, ejecútalo y compara tu salida con la de la guía. Para los bucles, ayuda mucho seguir el programa en papel, vuelta por vuelta.

TEMA	QUÉ DEBERÍAS PODER HACER AL TERMINAR
1. Listas	Crear una lista para guardar varios datos en una sola variable.
2. Posiciones	Acceder a un elemento por su posición, contando desde 0.
3. `len()` y `append()`	Contar los elementos de una lista y agregar otros nuevos.
4. Bucle `for`	Repetir una acción para cada elemento de una lista.
5. Acumuladores y contadores	Sumar y contar dentro de un bucle, y combinarlo con un `if`.
6. `range()`	Repetir una acción un número determinado de veces.
7. Funciones	Crear y llamar funciones con `def` y `return`.
8. Errores	Reconocer y corregir los errores más frecuentes de listas, bucles y funciones.

Al final encontrarás un **resumen**, un **glosario**, **ejercicios** para practicar y sus **soluciones**.

1 Listas

Con lo que sabemos hasta ahora, cada dato necesita su propia variable. Funciona con 4 notas, pero ¿y con 40? La solución es guardarlas todas juntas.

> **DEFINICIÓN**
> Una **lista** guarda **varios valores en orden** dentro de una sola variable. Cada valor de la lista se llama **elemento**.

Una variable por dato

```
nota1 = 3.5
nota2 = 4.2
nota3 = 2.8
nota4 = 4.9
```

Una lista

```
notas = [3.5, 4.2, 2.8, 4.9]
```

Crear una lista

Se escribe con **corchetes** `[ ]` y los elementos van separados por **comas**.

```
notas = [3.5, 4.2, 2.8, 4.9]
ciudades = ["Bogotá", "Medellín", "Cali"]
vacia = []

print(ciudades)
print(vacia)
```

SALIDA

```
['Bogotá', 'Medellín', 'Cali']
[]
```

- Puede guardar números, textos o cualquier otro tipo de dato.
- También puede empezar **vacía** e ir creciendo.
- `print()` muestra la lista completa, con corchetes y comillas.

2 Las posiciones empiezan en 0

DEFINICIÓN

Cada elemento tiene una **posición** (o **índice**). Para leerlo se escribe el nombre de la lista y la posición entre corchetes: `notas[0]`. El primer elemento está en la posición **0**.

POSICIÓN	0 (PRIMERO)	1	2	3 (ÚLTIMO)
Valor en `notas`	`3.5`	`4.2`	`2.8`	`4.9`

```
notas = [3.5, 4.2, 2.8, 4.9]
print(notas[0])
print(notas[3])
```

SALIDA

```
3.5
4.9
```

Una lista de 4 elementos tiene posiciones de 0 a 3. La posición 4 no existe:

```
notas = [3.5, 4.2, 2.8, 4.9]
print(notas[4])
```

```
IndexError: list index out of range
```

OJO

En Python se cuenta desde **0**. En R se cuenta desde **1**: lo veremos en la sesión 5. Es un detalle pequeño que causa muchos errores.

3 Contar y agregar: `len()` y `append()`

`len()` **cuenta**: cuántos elementos tiene la lista.

```
notas = [3.5, 4.2, 2.8, 4.9]
print(len(notas))
```

SALIDA

```
4
```

`.append()` **agrega**: pone un elemento al final.

```
notas = [3.5, 4.2, 2.8, 4.9]
notas.append(3.0)
print(notas)
print(len(notas))
```

SALIDA

```
[3.5, 4.2, 2.8, 4.9, 3.0]
5
```

Fíjate en cómo se escribe cada una: `len(notas)` lleva la lista entre paréntesis, mientras que `append` se escribe **después** de la lista, con un punto: es una acción que la lista sabe hacer sobre sí misma.

4 El bucle `for`

DEFINICIÓN

Un **bucle** repite un bloque de código. El bucle `for` lo repite **una vez por cada elemento** de una lista. Cada repetición se llama **vuelta** (o iteración).

```
notas = [3.5, 4.2, 2.8, 4.9]
for nota in notas:
    print("Nota:", nota)
```

SALIDA

```
Nota: 3.5
Nota: 4.2
Nota: 2.8
Nota: 4.9
```

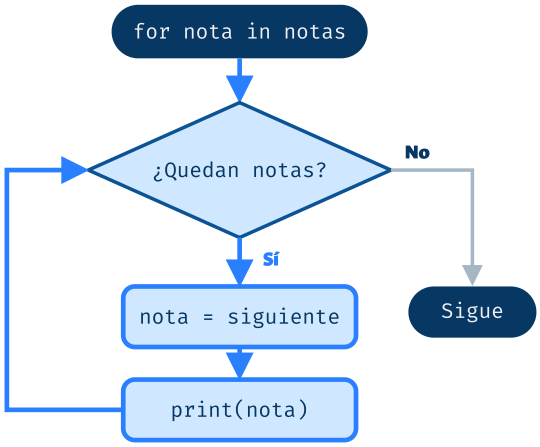
Se lee casi como en español: **"para cada nota en notas, imprime la nota"**.

Anatomía de un `for`

1. La palabra `for` ("para cada").
2. Una **variable** que toma cada valor, uno a la vez. El nombre lo eliges tú.
3. La **lista** que se recorre, después de `in`.
4. Los **dos puntos** y, debajo, el bloque **indentado** que se repite. Funcionan igual que en el `if`.

¿Cómo se ejecuta?

En cada vuelta, la variable `nota` cambia de valor. Cuando ya no quedan elementos, el programa sigue con la línea que hay después del bloque.



VUELTA	nota	IMPRIME
1	3.5	Nota: 3.5
2	4.2	Nota: 4.2
3	2.8	Nota: 2.8
4	4.9	Nota: 4.9

¿Dentro o fuera del bucle?

Igual que en el `if`, la indentación decide qué se repite:

Dentro: se repite

```
for n in [1, 2]:
    print(n)
    print("Listo")
```

SALIDA

```
1
Listo
2
Listo
```

Fuera: se ejecuta una vez, al final

```
for n in [1, 2]:
    print(n)
print("Listo")
```

SALIDA

```
1
2
Listo
```

5 Acumuladores y contadores

DEFINICIÓN

Un **acumulador** es una variable que va guardando un total: en cada vuelta del bucle le sumamos algo.

Tiene tres pasos: **crear** el acumulador antes del `for`, **actualizarlo** dentro y **usarlo** después.

```
notas = [3.5, 4.2, 2.8, 4.9]
suma = 0                      # 1. empieza en 0, antes del for
for nota in notas:
    suma = suma + nota        # 2. acumula en cada vuelta

promedio = suma / len(notas)  # 3. se usa al final
print("Promedio:", promedio)
```

SALIDA

```
Promedio: 3.85
```

Ejecución a mano

Este es el tema que más cuesta del curso, y la mejor forma de entenderlo es hacer esta tabla en papel, vuelta por vuelta:

VUELTA	nota	suma ANTES	OPERACIÓN	suma DESPUÉS
Inicio	—	—	`suma = 0`	`0`
1	`3.5`	`0`	`0 + 3.5`	`3.5`
2	`4.2`	`3.5`	`3.5 + 4.2`	`7.7`
3	`2.8`	`7.7`	`7.7 + 2.8`	`10.5`
4	`4.9`	`10.5`	`10.5 + 4.9`	`15.4`

Al final, `15.4 / 4` da `3.85`. Puedes verlo en Colab agregando un `print()` dentro del bucle:

```
notas = [3.5, 4.2, 2.8, 4.9]
suma = 0
for nota in notas:
    suma = suma + nota
    print("suma vale", suma)
```

SALIDA

```
suma vale 3.5
suma vale 7.7
suma vale 10.5
suma vale 15.4
```

OJO

El acumulador se crea **antes** del `for`, nunca dentro. Si escribes `suma = 0` dentro del bucle, vuelve a cero en cada vuelta y al final solo queda el último valor. Y usa `len(notas)` en lugar de escribir `4`: así el programa sigue funcionando cuando agregues una nota.

Bucle + condicional: contar

Aquí se juntan las sesiones 2 y 3. ¿Cuántos estudiantes aprobaron?

```
notas = [3.5, 4.2, 2.8, 4.9]
aprobados = 0
for nota in notas:
    if nota >= 3.0:
        aprobados = aprobados + 1

print("Aprobaron:", aprobados)
```

SALIDA

```
Aprobaron: 3
```

- El `if` va **dentro** del `for`: se revisa en cada vuelta.
- Hay dos niveles de indentación: 4 y 8 espacios.
- Un **contador** suma **1**; un acumulador suma **el valor**.

6 Repetir n veces con `range()`

DEFINICIÓN

`range()` genera una secuencia de números. Con un `for`, sirve para repetir algo un número de veces, aunque no tengamos una lista.

```
for i in range(5):
    print("Repetición", i)
```

SALIDA

```
Repetición 0
Repetición 1
Repetición 2
Repetición 3
Repetición 4
```

CÓDIGO	NÚMEROS QUE GENERA	PARA RECORDAR
`range(5)`	`0, 1, 2, 3, 4`	Da 5 números, pero empieza en 0.
`range(1, 6)`	`1, 2, 3, 4, 5`	Empieza en el primero y no incluye el último.

OJO

`range(5)` **no incluye el 5**. Si quieres contar de 1 a 5, escribe `range(1, 6)`.

Recorrer las posiciones de una lista

`range(len(notas))` da justo las posiciones de la lista. Sirve cuando necesitas saber en qué posición vas:

```
notas = [3.5, 4.2, 2.8]
for i in range(len(notas)):
    print("Posición", i, "tiene", notas[i])
```

SALIDA

```
Posición 0 tiene 3.5
Posición 1 tiene 4.2
Posición 2 tiene 2.8
```

7 Funciones

Ya usaste funciones sin crearlas: `print()`, `len()`, `type()`. Ahora vas a crear las tuyas. Si calculas el promedio de tres listas, ¿copias el mismo código tres veces?

> **DEFINICIÓN**
> Una **función** es como una receta con nombre: un bloque de código que se escribe una vez y se puede usar muchas veces.

- **Se escribe una vez:** la solución queda guardada con un nombre.
- **Se usa muchas veces:** con datos distintos cada vez.
- **Se corrige en un lugar:** si hay un error, se arregla una sola vez.

Anatomía de una función

```
def calcular_promedio(lista):
    suma = 0
    for valor in lista:
        suma = suma + valor
    return suma / len(lista)

print(calcular_promedio([3.5, 4.0, 4.5]))
print(calcular_promedio([2.0, 3.0]))
```

> **SALIDA**
> ```
> 4.0
> 2.5
> ```

PARTE	EN EL EJEMPLO	QUÉ HACE
`def`	`def`	"Definir": avisa que vamos a crear una función.
Nombre	`calcular_promedio`	Sigue las mismas reglas que los nombres de variables.
Parámetro	`lista`	El dato que la función recibe.
`return`	`return suma / len(lista)`	Devuelve el resultado a quien llamó la función.
Llamada	`calcular_promedio([3.5, 4.0, 4.5])`	Aquí se ejecuta de verdad. El valor que le pasamos se llama **argumento**.

> **OJO**
> **Definir no es ejecutar.** Si escribes el `def` y ejecutas la celda, no pasa nada visible: Python solo guarda la receta. El código corre cuando **llamas** a la función.

`print` muestra, `return` devuelve

Es la confusión más común con las funciones. `print` pone el resultado en pantalla, pero no lo entrega. `return` lo entrega para que puedas guardarlo y usarlo en otros cálculos.

`print` muestra

```
def saludar(nombre):
    print("Hola,", nombre)

r = saludar("Ana")
print(r)
```

SALIDA

```
Hola, Ana
None
```

`return` devuelve

```
def aplicar_iva(precio):
    return precio * 1.19

total = aplicar_iva(10000)
print(total)
```

SALIDA

```
11900.0
```

Si la función no tiene `return`, devuelve `None`, que significa “nada”.

Todo junto: lista, bucle, condicional y función

```
def contar_aprobados(notas):
    aprobados = 0
    for nota in notas:
        if nota >= 3.0:
            aprobados = aprobados + 1
    return aprobados

print(contar_aprobados([3.5, 4.2, 2.8, 4.9]))
print(contar_aprobados([1.0, 2.5]))
```

SALIDA

```
3
0
```

Adelanto: diccionarios

En una lista buscamos por **posición**; en un **diccionario**, por **nombre**. Es la idea de una fila de una tabla con nombres de columna, y la retomaremos en la próxima sesión. Por ahora, solo tienes que reconocerlo.

```
estudiante = {"nombre": "Ana", "edad": 20}
print(estudiante["nombre"])
```

SALIDA

```
Ana
```

8 Errores frecuentes

ERROR	CÓDIGO CON ERROR	QUÉ SIGNIFICA	CORRECCIÓN
`IndexError`	`notas = [3.5, 4.2]` `print(notas[2])`	Esa posición no existe. Con 2 elementos, la última es la 1.	`notas = [3.5, 4.2]` `print(notas[1])`

`NameError`	`for n in [1, 2]:`
	`  total = total + n`
		Al acumulador le falta su valor inicial.	`total = 0`
			`for n in [1, 2]:`
			`  total = total + n`

`SyntaxError`	`for n in [1, 2]` `  print(n)`	Al `for` le faltan los dos puntos, igual que al `if`.	`for n in [1, 2]:` `  print(n)`
`TypeError`	`for i in 5:` `  print(i)`	Un número no se puede recorrer: falta `range()`.	`for i in range(5):` `  print(i)`

`TypeError`	`def doble(x):`
	`  return x * 2`
	
	`print(doble())`
		Llamaste la función sin el dato que necesita.	`def doble(x):`
			`  return x * 2`
			
			`print(doble(4))`

Errores sin mensaje

Estos no muestran ningún error, pero el resultado está mal:

- **`suma = 0` dentro del `for`:** el acumulador se reinicia en cada vuelta.
- **`print` en lugar de `return`:** ves el resultado, pero la función devuelve `None` y no puedes usarlo.
- **`notas = notas.append(3.0)`:** `append` cambia la lista y devuelve `None`, así que pierdes tu lista. Basta con `notas.append(3.0)`.

Según la versión de Python, el texto de algunos mensajes puede cambiar un poco, pero el tipo de error es el mismo.

Tres ideas clave

1. Las listas guardan varios datos en orden; en Python la primera posición es 0.
2. El for repite una acción para cada elemento.
3. Las funciones empaquetan una solución para reutilizarla.

Glosario

Acumulador	Variable que va guardando un total; en cada vuelta se le suma algo.
`append()`	Agrega un elemento al final de una lista.
Argumento	Valor que se le pasa a una función al llamarla.
Bucle	Instrucción que repite un bloque de código.
Contador	Variable a la que se le suma 1 cada vez que ocurre algo.
`def`	Palabra con la que se crea una función.
Diccionario	Colección de datos a los que se accede por nombre, no por posición.
Elemento	Cada uno de los valores guardados en una lista.
`for`	Bucle que repite un bloque una vez por cada elemento.
Función	Bloque de código con nombre que se escribe una vez y se usa muchas veces.
Índice	Posición de un elemento dentro de una lista. Empieza en 0.
`len()`	Dice cuántos elementos tiene una lista.
Lista	Colección ordenada de valores dentro de una sola variable.
Llamar	Ejecutar una función escribiendo su nombre y sus paréntesis.
`None`	Valor que significa “nada”. Es lo que devuelve una función sin `return`.
Parámetro	Nombre que recibe, dentro de la función, el dato que le pasan.
`range()`	Genera una secuencia de números para repetir algo varias veces.
`return`	Termina la función y entrega un resultado.
Vuelta	Cada repetición de un bucle. También se llama iteración.

Ejercicios

Resuélvelos en Colab antes de mirar las soluciones. Avanza hasta donde llegues.

Ponte a prueba: ¿qué imprime cada programa?

Escribe tu respuesta **antes** de ejecutar el código.

CÓDIGO	TU RESPUESTA
`notas = [2.0, 3.5, 4.0]` `print(notas[1])`	

`notas = [2.0, 3.5, 4.0]`
`notas.append(5.0)`
`print(len(notas))`	

`for i in range(3):` `    print(i)`	

`suma = 0`
`for n in [1, 2, 3]:`
`    suma = suma + n`
`print(suma)`	

`def doble(x):`
`    return x * 2`
 	
`print(doble(4))`	

`def doble(x):`
`    print(x * 2)`
 	
`r = doble(4)`
`print(r)`	

Encuentra el error

Di qué tipo de error produce cada programa y corrígelo.

CÓDIGO	ERROR Y CORRECCIÓN
`ciudades = ["Bogotá", "Cali"]` `print(ciudades[2])`	
`for ciudad in ["Bogotá", "Cali"]` `    print(ciudad)`	
`for n in [1, 2]:` `print(n)`	

Completa el código: el promedio de mis notas

Llena los espacios para calcular el promedio de la lista. Luego agrega una nota más con `append()` y vuelve a ejecutarlo.

```
notas = [4.0, 3.5, 4.5, 3.0]
suma = ______
for nota in notas:
    suma = ______
print("Promedio:", suma / ______)
```

Ejecución a mano

Completa la tabla sin ejecutar el código. ¿Cuánto vale `suma` al terminar cada vuelta?

```
suma = 0
for nota in [2.0, 4.0, 3.0]:
    suma = suma + nota
```

VUELTA	nota	suma ANTES	suma DESPUÉS
1			
2			
3			

Ejercicios de la clase

BÁSICO **1.** Crea una lista con 5 ciudades de Colombia e imprime cada una con un bucle.

BÁSICO **2.** Dada una lista de precios, calcula el total de la compra.

INTERMEDIO **3.** Cuenta cuántas notas de una lista son mayores o iguales a 3.0.

INTERMEDIO **4.** Escribe una función `aplicar_iva(precio)` que devuelva el precio con IVA del 19 % y úsala sobre una lista de precios.

RETO **5.** Encuentra la nota más alta de una lista **sin** usar `max()`.

RETO **6.** Escribe una función que reciba una frase y devuelva cuántas palabras tiene. Pista: `frase.split()`.

> CONSEJO
> ¿Te atascaste? Haz la **tabla de vueltas** a mano. Para el ejercicio 5, guarda la primera nota como "la más alta hasta ahora" y compárala con las demás dentro del `for`.

Soluciones

Hay muchas formas correctas de resolver cada ejercicio. Si tu código es distinto pero da el resultado correcto, también está bien.

Ponte a prueba

CÓDIGO	SALIDA	POR QUÉ
`notas = [2.0, 3.5, 4.0]` `print(notas[1])`	`3.5`	La posición 1 es el **segundo** elemento.

`notas = [2.0, 3.5, 4.0]`
`notas.append(5.0)`
`print(len(notas))`
	`4`	Tenía 3 elementos y se agregó uno.

`for i in range(3):`
`  print(i)`
	`0`
	`1`
	`2`
		Empieza en 0 y no incluye el 3.

`suma = 0`
`for n in [1, 2, 3]:`
`  suma = suma + n`
`print(suma)`
	`6`	0 + 1 + 2 + 3. El `print` está fuera del bucle: se ejecuta una vez.

`def doble(x):`
`  return x * 2`
 	
`print(doble(4))`
	`8`	La función devuelve 8 y `print` lo muestra.

`def doble(x):`
`  print(x * 2)`
 	
`r = doble(4)`
`print(r)`
	`8`
	`None`
		La función muestra 8, pero no tiene `return`: devuelve `None`.

Encuentra el error

CÓDIGO	ERROR	CORRECCIÓN
`ciudades = ["Bogotá", "Cali"]` `print(ciudades[2])`	`IndexError`: las posiciones son 0 y 1.	`ciudades = ["Bogotá", "Cali"]` `print(ciudades[1])`
`for ciudad in ["Bogotá", "Cali"]` `  print(ciudad)`	`SyntaxError`: faltan los dos puntos.	`for ciudad in ["Bogotá", "Cali"]:` `  print(ciudad)`
`for n in [1, 2]:` `print(n)`	`IndentationError`: falta la sangría.	`for n in [1, 2]:` `  print(n)`

Completa el código

```
notas = [4.0, 3.5, 4.5, 3.0]
suma = 0
for nota in notas:
    suma = suma + nota
print("Promedio:", suma / len(notas))
```

SALIDA

```
Promedio: 3.75
```

Si escribiste `4` en lugar de `len(notas)`, el resultado es correcto al principio, pero falla cuando agregas una nota.

Ejecución a mano

VUELTA	nota	suma ANTES	suma DESPUÉS
1	`2.0`	`0`	`2.0`
2	`4.0`	`2.0`	`6.0`
3	`3.0`	`6.0`	`9.0`

Próxima sesión: primer contacto con datos reales, con pandas. ¿Y si la lista tuviera miles de filas? Usaremos un archivo CSV que se compartirá por enlace. Trae tus dudas de esta guía a la clase o escríbelas en el Moodle FCE.